

How iHIS Was Built

One patient, one record, one connected hospital — with artificial intelligence built into the workflow, not bolted onto it. A five-page account of the vision, the architecture, the clinical workflows, the AI layer, and the engineering process (29 Aug – 11 Sep 2026).

1. The problem and the idea

Hospitals run on fragmented systems: reception, physicians, laboratory, radiology, pharmacy, nursing and billing each keep their own view of the patient, and the "AI" offered to them is usually a chat window that knows nothing about the chart. iHIS started from a different premise: **the record is the product**, and AI is valuable only when it reads that record, explains itself, and hands every decision back to a licensed clinician.

The system was therefore designed around four rules that never changed during development:

Rules first, AI explains

Critical findings, medication safety, early-warning scores and risk profiles run on deterministic engines. A language model may elaborate on them, but it can never contradict them, create a diagnosis, or write to the chart.

Nothing fake

Every AI feature shown as "available" really runs on this server. Anything not installed says "coming soon". Every number on screen is traceable to a record, a rule or a model output.

Need-to-know access

A physician sees the patients they treat; a dermatologist sees the dermatology model, not the chest X-ray one. Access is documented, and every action is audited.

Local and small

All image models run on the machine's CPU, each under 500 MB; images and reports never leave the server. The language model is an optional add-on with a plan B.

What it became

13

CLINICAL & ADMIN ROLES

91

DATABASE TABLES

313

WEB ROUTES

5

LOCAL IMAGING MODELS

175

SCREENS (TEMPLATES)

409

AUTOMATED TESTS

The whole system — records, appointments, laboratory, imaging, pharmacy, nursing, admissions, dentistry, physiotherapy, billing, reports and the AI layer — is a single Python application of about 26,600 lines, built in fourteen days across fifty commits, and it runs on a laptop without Docker, cloud services or a GPU.

2. Architecture and technology

iHIS is a server-rendered web application: Python 3 with **Flask 3** (application factory, blueprints per department), **SQLAlchemy** models with **Alembic** migrations, Jinja2 templates with a bilingual English/Arabic layer (right-to-left aware), Flask-Login sessions, CSRF protection, rate limiting and structured audit logging. SQLite is used locally; PostgreSQL is a configuration change.

The data model

Ninety-one tables describe the hospital: users, roles and fine-grained permissions; patients with a generated medical record number; appointments and queues; encounters with diagnoses (ICD-10-CM), structured problem lists and allergies; laboratory orders, specimens and verified results with critical thresholds; radiology orders, reports with versions, dose records and a critical-finding register; prescriptions, dispensing, inventory batches, reconciliations and pharmacist interventions; vital signs, nursing notes, medication administration; wards, beds and admissions; dental charts and therapy plans; bills and payments; tasks, notifications, a cross-department timeline, clinical alerts with a full lifecycle, and an AI usage log.

Roles and access

Thirteen roles (Super Administrator, Administrator, Physician, Nurse, Laboratory Technician, Radiologist, Radiology Technician, Pharmacist, Receptionist, Dentist, Physiotherapist, Cashier, Patient) map to permissions, and permissions gate routes. On top of role checks sits a **need-to-know** layer: a staff member can open a patient only through a documented relationship — an appointment, an order, an admission, a referral, a care-team membership or a department assignment. Patient lists, AI tools and document access all flow through the same rule, so a filtered list never leads to a forbidden page.

Reference data that makes the rules real

Reference	Size	Used for
ICD-10-CM 2026 (CDC, public domain)	74,719 codes	Diagnosis entry with local-first autocomplete
DDInter drug–drug interactions (CC BY-NC-SA 4.0)	160,235 pairs	Interaction checks under the local formulary
openFDA drug labels + RxNorm	live, cached 12 h	Official label sections and identifiers per medication
FDA / SmPC renal-dosing thresholds (curated)	~35 drugs	Dose rules by creatinine clearance
NEWS2, qSOFA, LACE, Charlson-style comorbidity	published scores	Early warning, sepsis screen, readmission, risk profile

Design system

One visual language across all 175 screens: green is the hospital, a violet-to-blue gradient is reserved for AI, so anyone can tell at a glance what a model produced and what the record holds. Every role home follows the same shape — a few numbers, one AI block with the local models as large cards, then the day's work. The interface passed a WCAG 2.1 AA scan and real-browser journey tests, and every screen exists in English and Arabic.

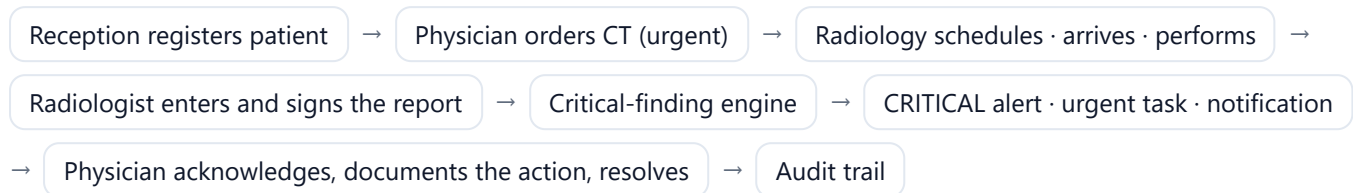
3. Clinical workflows, end to end

Each department got a complete, auditable workflow before any AI was attached to it. Three of them show how the pieces connect.

Radiology critical finding

rules

AI-assisted



The engine reads every signed report with clinical rules (pulmonary embolism, pneumothorax, aortic dissection, intracranial haemorrhage and more) plus a local negation classifier so "no evidence of pneumothorax" is not flagged. The alert is assigned to the ordering physician, appears as a red banner on their home, and is mirrored in the radiology department's critical-finding register; acknowledging one acknowledges the other. The whole chain was exercised through the user interface, click by click, with a synthetic CT pulmonary angiography case.

Clinical pharmacist review

rules

When a prescription is written, a deterministic review runs against the entire chart, not only the lines on that prescription: renal function estimated by Cockcroft-Gault from the latest creatinine, weight, age and sex; renal dose rules from drug labels; interactions against every other active medication (formulary pairs with mechanism and management first, DDIInter as the reference layer); label contraindications such as colchicine with a P-gp / CYP3A4 inhibitor in renal impairment; drug-laboratory conflicts such as an ACE inhibitor with a high potassium; allergies (with "NKDA" recognised) and duplicates. Each finding raises its own alert, and the pharmacist's prescription page shows a review card with the renal panel, the findings with sources, and one-click pre-filled interventions to the prescriber. The demo patient — a 68-year-old on apixaban and diltiazem with stage 3b kidney disease receiving a colchicine loading dose — returns a verdict of CRITICAL with prednisone suggested as the alternative, exactly as a clinical pharmacist would write it.

Dermatology outpatient clinic

local model

Two synthetic cases prepared by the dermatologist: a changing shoulder mole with a positive ABCDE assessment and a stable, uniform mole on the back. Each patient is registered under Dermatology, seen in clinic, documented in a signed note, and has a real dermoscopic image (ISIC archive, public domain) attached to the chart. The skin-lesion model is run on the attached image through the same code the page uses; the first case returns melanoma at 98 % with a Grad-CAM heatmap and produces a critical alert, an urgent task, a biopsy order and a lymph-node referral; the second returns a benign nevus at 99.9 % and produces nothing but a baseline record and a twelve-month follow-up. Nothing in either outcome is hard-coded.

Everything else

- **Laboratory:** order → specimen → result → verification, with critical values escalating to the physician.
- **Nursing:** vitals feed NEWS2 and qSOFA automatically; medication administration, care plans, fluid balance.
- **Admissions:** atomic bed claims, transfers, discharge summaries; inpatient and outpatient views split on every list.
- **Pharmacy operations:** FEFO batch dispensing, partial dispensing, expiry and stock alerts, reconciliation at transitions of care.
- **Front desk and finance:** registration, booking, queues, bills generated from clinical events, receipts and reports.
- **Patient portal:** own record, appointments, results explained in plain language by a safe patient AI that never diagnoses.

4. The AI layer

AI in iHIS has three tiers, in decreasing order of authority: deterministic engines, local models, and an optional language model. The tiers are visible in the interface — a "RULES" or "MODEL" tag on one side, an "AI-ASSISTED · NOT A FINAL DIAGNOSIS" tag on the other.

Five local imaging models (all under 500 MB, CPU only)

Model	Engine	Output	Who sees it
Chest X-ray Screening	TorchXRayVision DenseNet-121 (~30 MB)	18 findings with probabilities; confident critical ones alert	Radiology, internal medicine, cardiology, pulmonology...
Fracture Detection	YOLOv8	Annotated bone X-ray	Radiology, orthopaedics, emergency, surgery
Tooth Segmentation	U-Net	Segmentation mask on panoramic X-rays	Dentistry, radiology
Skin Lesion Detection	ResNet-50 + EfficientNet-B0 ensemble	Melanoma vs nevus with Grad-CAM; image-quality check	Dermatology, family medicine, oncology
Head CT Haemorrhage Detection	ConvNeXt-Tiny + Swin-Tiny (~107 MB each)	Slice or whole-series calls on DICOM windows with LayerCAM	Radiology, emergency, neurology, neurosurgery

A single **specialty policy** decides which models each user sees and can open; it is applied identically to the sidebar, the dashboard block, the AI hub and the routes themselves, and it follows the physician's specialty on file. Every run writes an audit row, a clinician records an explicit review, and positive findings on patient-linked images enter the same alert lifecycle as a critical laboratory value.

The Clinical Copilot and the language-model providers

One assistant is reachable from every clinical page. It reads verified chart data first — summaries, differentials, documentation drafts, safety review, patient communication — and calls the language model only on an explicit click, with the deterministic findings passed in as an authoritative block it may not contradict. Prompts are wrapped in a guard that treats patient text strictly as data, and instruction-like content in the record is flagged rather than obeyed. Google Gemini is the primary provider; **Groq** is the automatic plan B when Gemini is missing, over budget, cooling down after a rate limit, or failing — same prompt, same guard, same audit row with the provider named. A per-minute and per-day budget, a cache, and a control centre for usage, failures and latency keep the free tiers within limits. Local dictation (faster-whisper) sits next to every clinical note.

Explainable risk, not a black box

The patient **Health Insights** page replaced an early heuristic with a transparent profile: age, Charlson-style comorbidity, kidney function, potassium, recent abnormal and critical laboratory results, NEWS2, polypharmacy and high-risk drug classes, pharmacist findings, documented allergies, admissions and open alerts. Every factor shows its points, its evidence and its source; data gaps are listed; and the optional AI narrative explains the factors without changing the score.

The recurring design test throughout was a single question: **if a judge clicks this, can we show exactly where the number came from?** Anything that failed the test was rewritten or removed — including an early "AI alert scanner" and a demo page whose scenarios pointed at whatever record happened to be newest.

5. How the work was done, and what is next

An AI-assisted engineering process

iHIS was built by a small team working with **Claude Code** as an engineering partner, in short missions with explicit rules: work autonomously, never claim a feature that is not implemented, never fabricate a prediction or an alert, never spend live language-model quota inside the test suite, never expose secrets, and never claim something was verified unless it actually was. Each mission ended with the same checklist — tests green, the change exercised in a real browser, documentation updated, commit pushed. The result is fifty commits over fourteen days, each one describing what changed and why.

Quality gates

409 automated tests (55 files)

Unit and route tests with mocked models and providers: access matrices per role, alert lifecycles, dose rules, provider fallback, media privacy, page rendering health for every role.

Real-browser journeys (Playwright)

Login, ordering, reporting, signing, acknowledging — driven through the actual interface with screenshots at every step; the radiology critical-finding case was created entirely by clicking.

Accessibility and language

WCAG 2.1 AA scan with axe, keyboard navigation, contrast tokens, ARIA labels; a full Arabic layer audited string by string.

Link and route crawl

Every link and form for all thirteen roles crawled; zero broken pages; duplicates retired with redirects.

Demo and deployment

The project stays on a local machine by choice: no Docker, no GPU, no paid hosting. For a presentation, a one-command script opens a Cloudflare tunnel from the laptop and prints a public HTTPS link, so an audience can use every feature — including the image models — from their phones. Free cloud tiers were evaluated and documented honestly: none of them can host the model stack for free, and Hugging Face Spaces becomes an option once the account meets its age requirement. Three prepared demo patients (a pharmacology case, two dermatology cases) and a click-through radiology case, plus illustrated Arabic role guides and a judging walkthrough, make the demonstration repeatable in minutes.

Known limits

- The imaging models are research-grade decision support, not medical devices; thresholds and labels come from their authors' test sets, and the interface says so on every result.
- The head-CT sequence model (LSTM stage) is not installed; series are scored by the convolutional model alone until the checkpoint is added.
- DDInter is licensed for non-commercial use; a commercial deployment would need a licensed interaction source.
- The built-in development server and SQLite are suitable for demonstration; production needs a WSGI server and PostgreSQL, both already supported by configuration.

What comes next

The natural extensions are the ones the architecture already anticipates: a real head-CT series for the fifth model, local accuracy monitoring of each model on the hospital's own confirmed cases, FHIR export of the record (a read-only FHIR surface exists), and a grounded clinical-reasoning engine in which every generated claim is checked against the chart before it is shown. Each of these builds on the same principle that shaped everything else in iHIS: **the clinician decides; the system makes sure the decision is informed, traceable and safe.**